

Certains capteurs météorologiques sont capables de faire des relevés très précis de certaines valeurs comme la température, la pression atmosphérique etc.

On propose d'étudier un capteur de température qui est capable de relever des valeurs de températures à des intervalles réguliers (toutes les heures). Ce capteur va créer un fichier texte nommé `meteo.txt`.

Le format de sortie du fichier `meteo.txt` pourrait ressembler à:

```
jour,heure,temperature
2025-06-01,00,14.8
2025-06-01,01,14.3
2025-06-01,02,13.9
2025-06-01,03,13.7
2025-06-01,04,13.9
2025-06-01,05,15.2
2025-06-01,06,17.5
2025-06-01,07,19.6
2025-06-01,08,21.8
```

On se propose ici d'étudier une application capable de:

- récupérer, ouvrir et convertir le fichier `meteo.txt`,
- calculer les moyennes de températures glissantes sur trois heures consécutives,
- déterminer les températures maximales et minimales relevées,
- détecter des pics de chaleur,
- détecter les risques de gel.

Une fois ouvert et convertit, les données seront stockées dans une liste de dictionnaires `mesures` du type:

```
</> Code Python
1 [{"jour": "2025-06-01", "heure": "00", "temperature": 14.8},
2  {"jour": "2025-06-01", "heure": "01", "temperature": 14.3},
3  {"jour": "2025-06-01", "heure": "02", "temperature": 13.9},
4  {"jour": "2025-06-01", "heure": "03", "temperature": 13.7},
5  {"jour": "2025-06-01", "heure": "04", "temperature": 13.9},
6  {"jour": "2025-06-01", "heure": "05", "temperature": 15.2},
7  {"jour": "2025-06-01", "heure": "06", "temperature": 17.5},
8  {"jour": "2025-06-01", "heure": "07", "temperature": 19.6},
9  {"jour": "2025-06-01", "heure": "08", "temperature": 21.8}]
```

1. Ecrire une fonction `extraire_temperatures(mesures)` qui prend une liste de dictionnaires de mesures en paramètres et renvoie une liste dont on n'aura gardé que les températures. On conservera les températures dans l'ordre temporel.

```
</> Code Python
1 def extraire_temperatures(mesures):
2     ''' Renvoie une liste de températures dans l'ordre chronologique
3     mesures: liste de dictionnaires contenant les mesures
4     sortie: liste de températures
5     '''
6     liste_temperatures = []
7     ### A compléter #####
8
9     #####
10    return liste_temperatures
```

On pourra tester cette méthode en décommentant la ligne `#test_extraire_temperatures()`.

2. Ecrire une fonction `detection_pic_chaleur(liste_temp, seuil)` qui prend en paramètre une liste de températures de type `list` et renvoie une liste contenant les jour et heure où les températures auront dépassé un certain seuil.

On pourra par exemple avoir le comportement suivant:

Code Python

```
1 liste_temp = [14.8, 14.3, 13.9, 13.7, 13.9, 15.2, 17.5, 19.6, 21.8, 23.1, 25.0, 26.2,
2               27.5, 28.3, 28.8, 27.9, 26.0, 24.2, 21.9, 19.4, 17.1, 15.6, 15.0, 14.4]
3 seuil = 28
4
5 pics = detection_pic_chaleur(liste_temp, seuil)
6 print(pics)
7 # Résultat attendu: [('2025-06-01', '13'), ('2025-06-01', '14')]
```

On pourra tester cette méthode en décommentant la ligne `#test_detection_pic_chaleur()`.

3. Les températures brutes heure par heure ne sont pas forcément représentatives des températures sur une journée. Il est plus intéressant de connaître la moyenne de température glissante sur des périodes de trois heures.

Ainsi si les températures relevées sont: [14, 18, 22, 20, 15, 13], on va calculer les moyennes de température sur:

- les valeurs 1, 2, 3: moyenne de 14, 18 et 22, soit 18.0
- les valeurs 2, 3, 4: moyenne de 18, 22 et 20, soit 20.0
- les valeurs 3, 4, 5: moyenne de 22, 20 et 15, soit 19.0
- les valeurs 4, 5, 6: moyenne de 20, 15 et 13, soit 16.0

La valeur renvoyée sera donc une liste contenant les quatre moyennes glissantes sur trois températures: [18.0, 20.0, 19.0, 16.0]

Un programme récursif a été écrit. Il ne donne cependant pas les bons résultats. Identifier la ou les erreur(s) et proposer une correction.

Code Python

```
1 def moyennes_glissantes(liste):
2     """
3     Calcule récursivement la moyenne glissante de 3 valeurs consécutives dans la liste.
4     """
5     ### A modifier #####
6     if len(liste) == 3:
7         return []
8
9     moyenne = (liste[0] + liste[1] + liste[2]) / 3
10
11     nouvelle_liste = []
12     for i in range(1, 4):
13         nouvelle_liste.append(liste[i])
14     return [moyenne] + moyennes_glissantes(nouvelle_liste)
15 #####
```

On pourra avoir un résultat du type:

Code Python

```
1 liste_temp = [14, 18, 22, 20, 15, 13]
2
3 m_glissantes = moyennes_glissantes(liste_temp)
4 print(m_glissantes)
5 # Résultat attendu: [18.0, 20.0, 19.0, 16.0]
```

On pourra tester cette méthode en décommentant la ligne `#test_moyennes_glissantes()`.